

Single Layer Perceptron

- $x \in \mathbb{R}^d \equiv$ input sample

$w \in \mathbb{R}^d \equiv$ weight matrix

$y \in \mathbb{R} \equiv$ output

$$y = \varphi(r) \text{ where } r = w^T x + w_0$$

$$\uparrow \varphi(r) = \begin{cases} -1 & r \leq 0 \\ 1 & r > 0 \end{cases} \leftarrow$$

$y \equiv$ predicted label

heaviside or sign function

$r = w^T x + w_0 \rightarrow$ creating a hyperplane that divides our data

where, this hyperplane is defined by $w^T x + w_0 = 0$

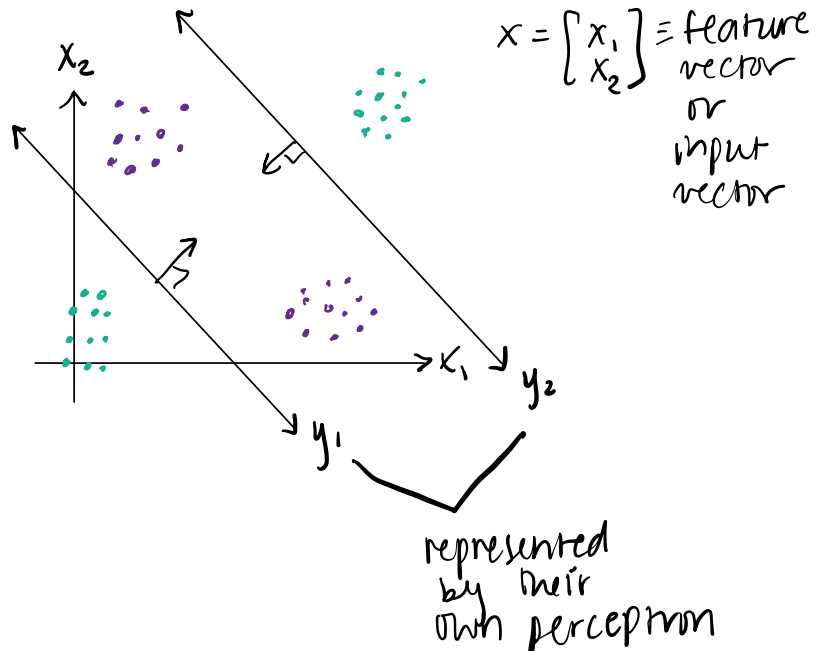
$\leq 0 \rightarrow$ we have class 1

$> 0 \rightarrow$ we have class 2

- example :

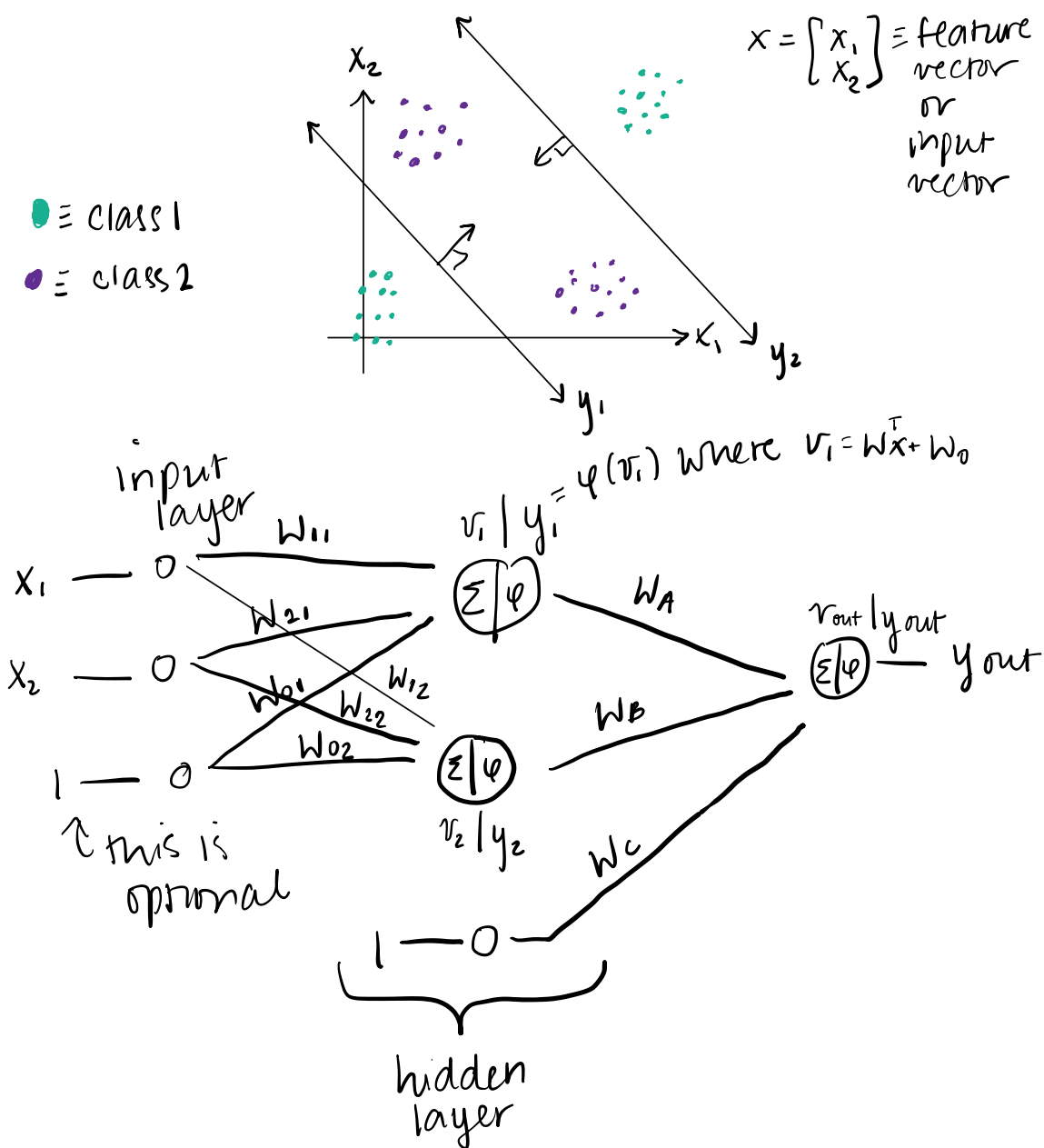
● $\equiv$ class 1

● $\equiv$ class 2



solution: multilayer perceptrons (MLPs)

how it works: stack perceptrons



$$y_1 = \varphi(W_{11}x_1 + W_{21}x_2 + W_{01})$$

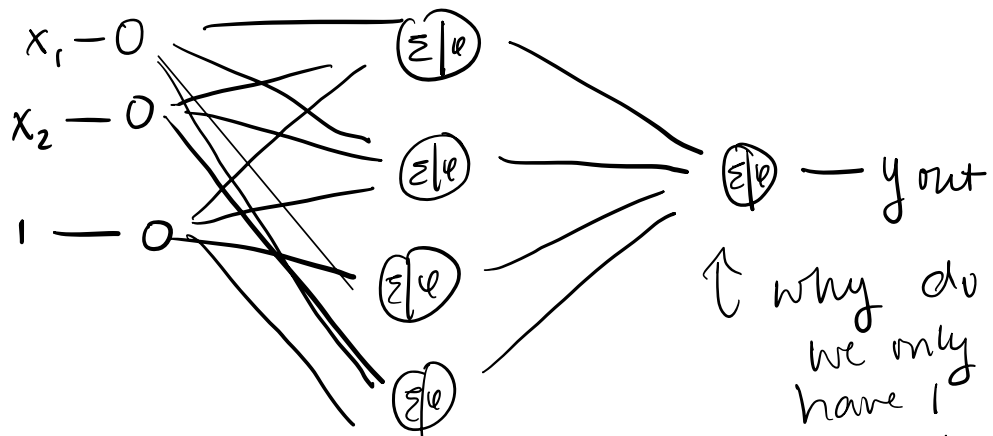
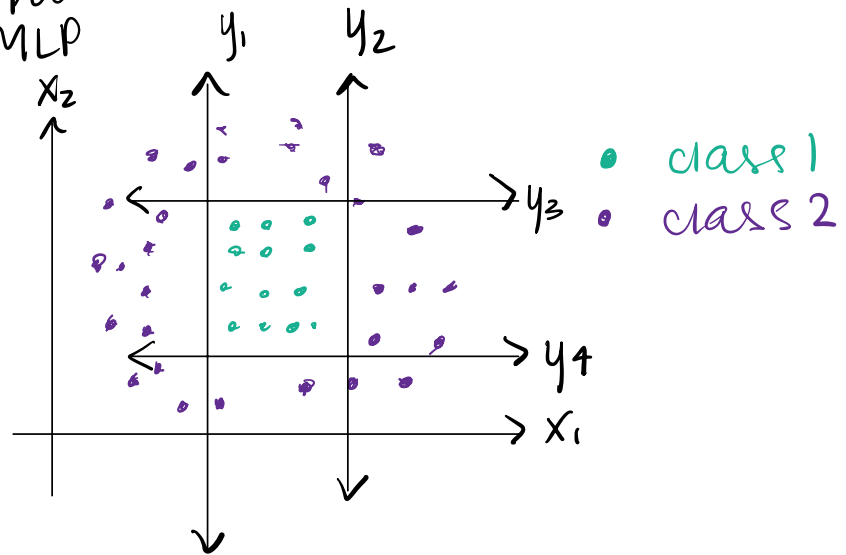
$$y_2 = \varphi(W_{12}x_1 + W_{22}x_2 + W_{02})$$

↑ hidden node
input channel

$$y_{out} = \varphi(W_A y_1 + W_B y_2 + W_C)$$

↑ output of MLP

- next example



↑ why do we only have 1 output node?

binary classification problem

what if we had more classes?

① need as many output nodes as number of classes with

one hot : categorical
encoding labels
of targets 1, 2, and 3

1: [1, 0, 0]

2: [0, 1, 0]

3: [0, 0, 1]

target $\Rightarrow$ needs to
be the
length =
highest
categorical
label

softmax :
activation
on output
 $\sigma(x_n) = \frac{e^{x_n}}{\sum_m e^{x_m}}$ $\leftarrow$ represent the channel
pseudo probability
 $\uparrow$
iterating over output channels

- Sum to one
- non-negativity

loss $\rightarrow$ cross entropy (standard)
 $CE(p, q) = -p \log(q)$

or MSE (not as common)

② one output neuron $\rightarrow$ outputs as categorical

activation function $\rightarrow$ linear

* ReLU

$\uparrow$ rectified

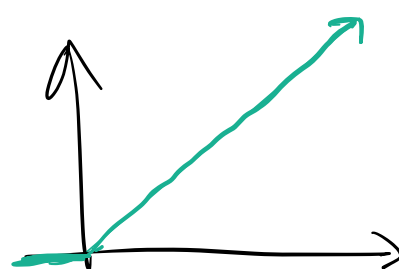
* GELU $\leftarrow$ gaussian linear unit

linear unit

* ELU $\leftarrow$ exponential linear unit

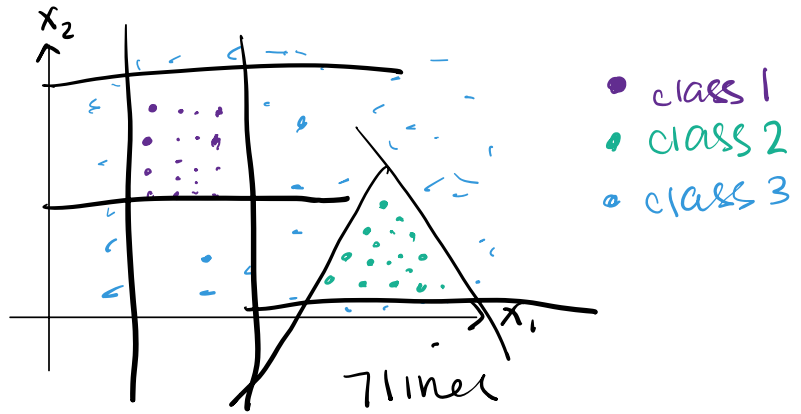
ReLU $\rightarrow$ most commonly used linear activation function

$$\psi(x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

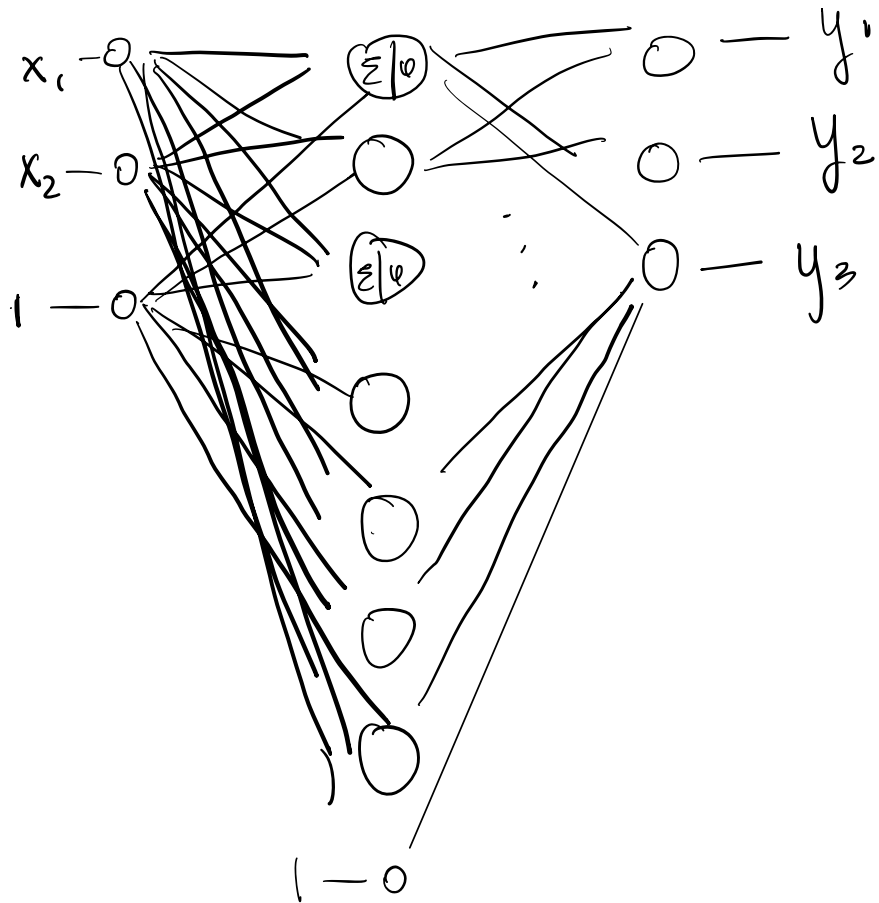


• ReLU

- example:



three
node:



$$\vec{y} = [y_1, y_2, y_3] = [0.1, -1, 0.9]$$

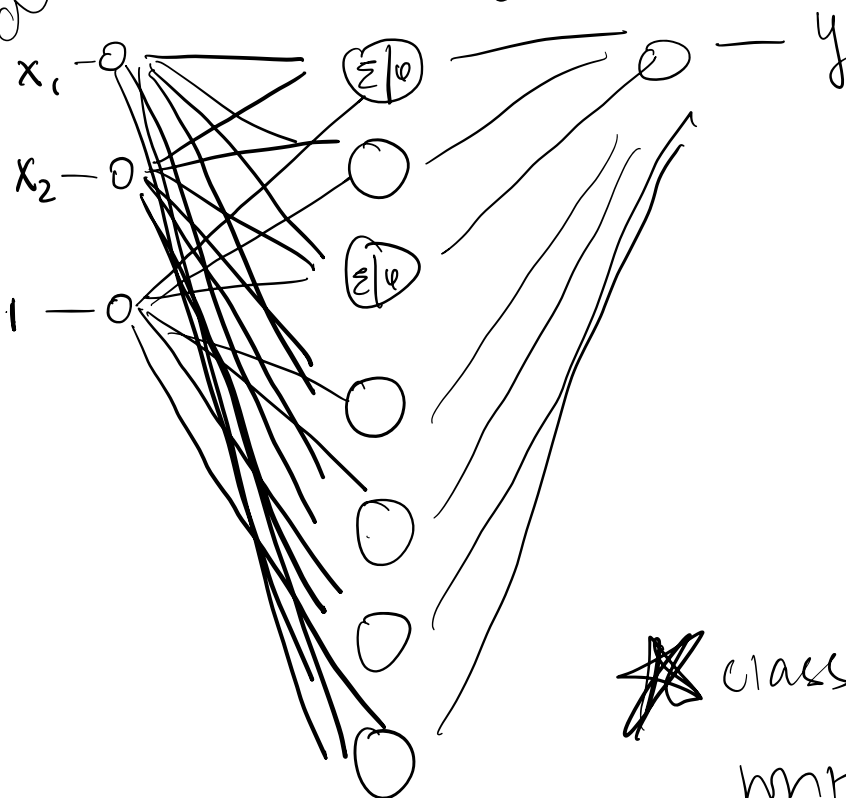
$$\vec{t} = [0, 0, 1]$$

$$CE(p, q) = - \sum_{c=1}^C p_c \log(q_c)$$

$$= - (0 \log(.1) + 0 \log(.1) + 1 \log(.8))$$

$$= - \log(.8)$$

one node:



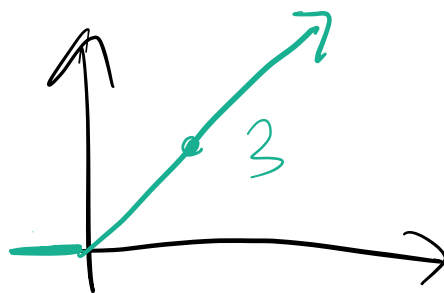
softmax
maxes
out at 1

$$y = 2.3 \quad t = 3$$

↑ softmax can't reach

* classes could
be
written or
encoded as
0, 1, 2 or
1, 2, 3 for a
three class
problem

rem



loss is no longer

CE

$$\text{MSE} \quad \|t - y\|_2^2$$
$$\frac{1}{N} \sum_{n=1}^N \|t_n - y_n\|_2^2$$

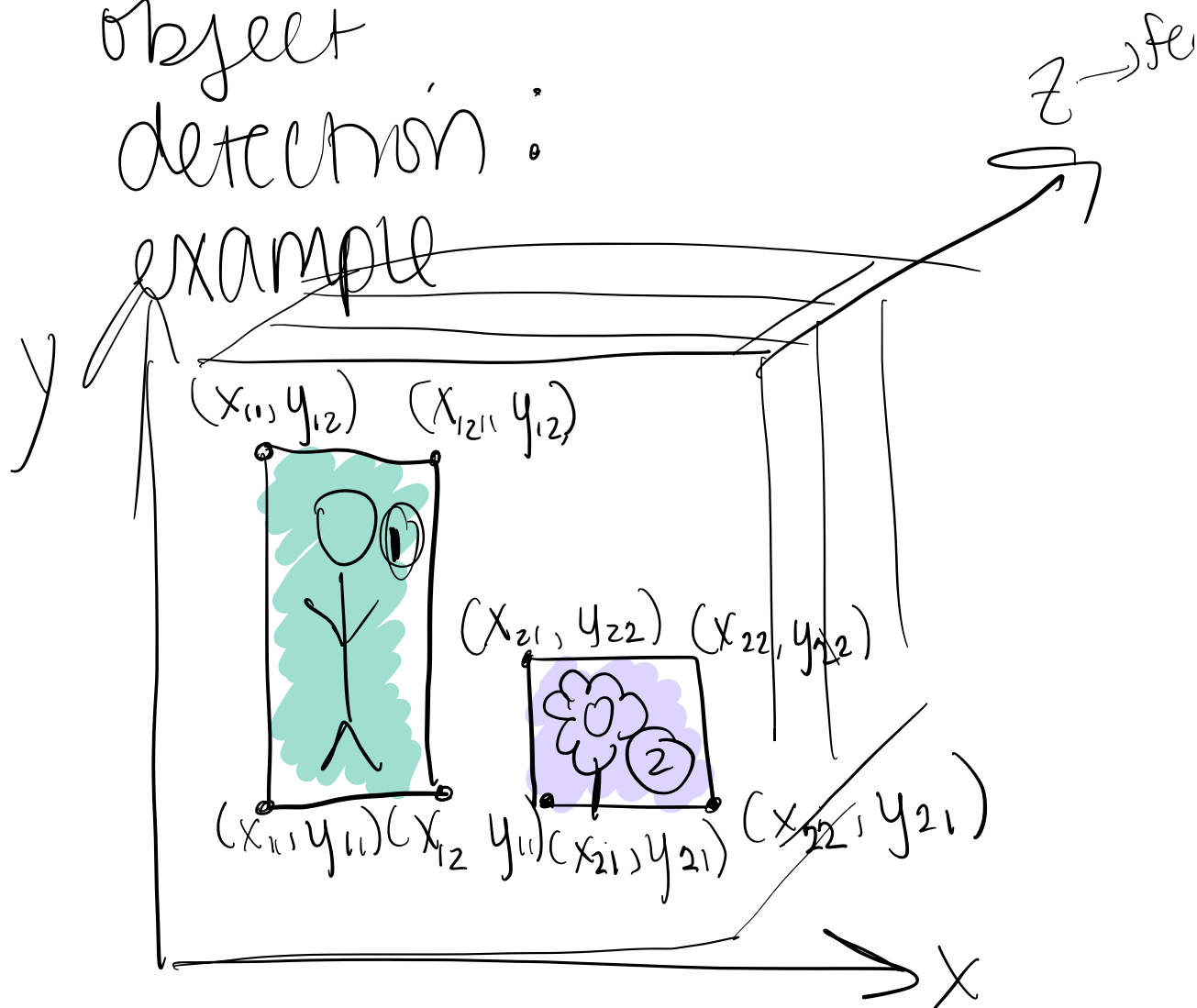
regression
task

object
detection

- ① classification task
- ② regression task

object
detection:

example



regression
task

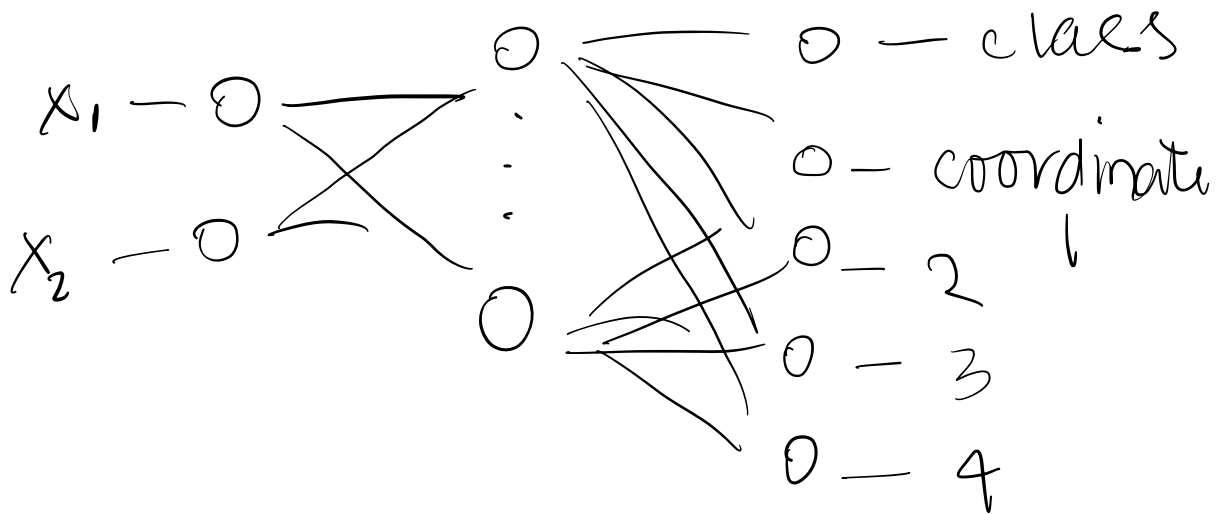
→ localizing
the object

predicting
the continuous
values in
spatial / pixel

space

classification
task

identifying
type of
object



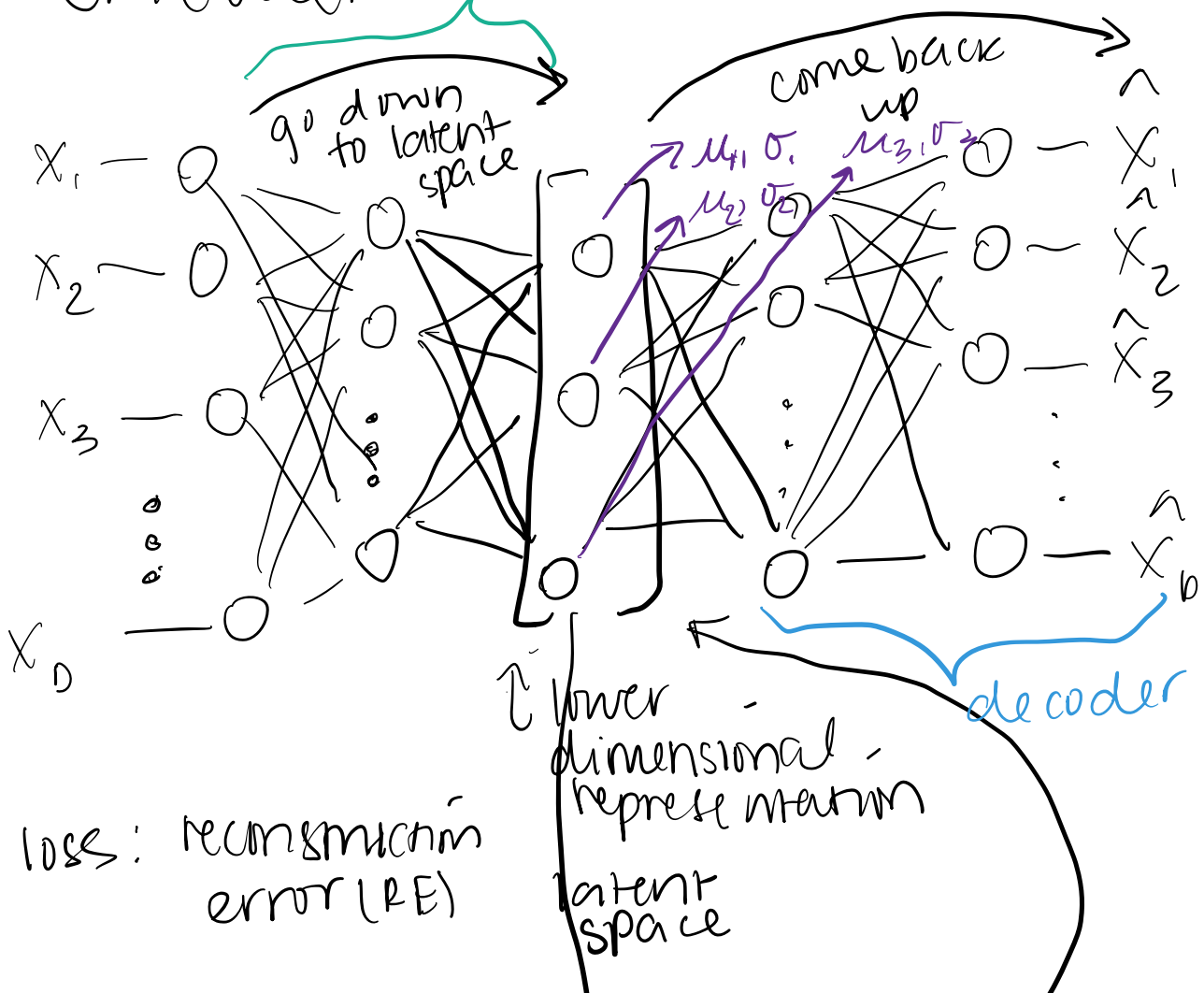
Universal
Approximation
Theorem

an MLP with
one hidden
layer can
approximate
arbitrarily

close any
continuous
function"

Auto
encoder

regression task
"reconstruct"



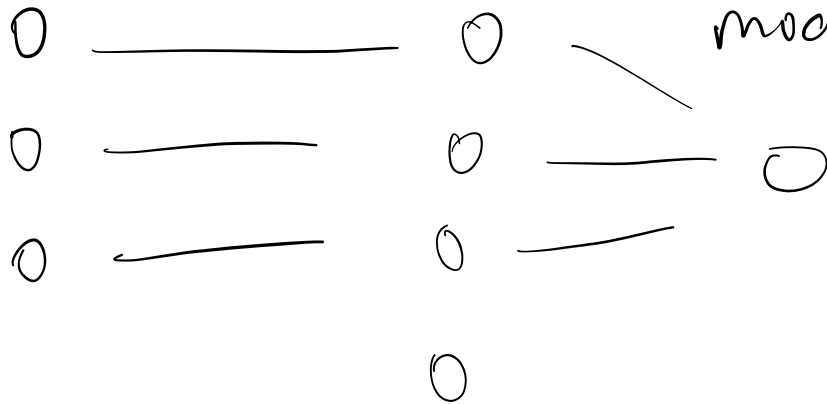
$$RE = \|x - \hat{x}\|_2^2$$

VAE - variational
auto encoders

learn a
distribution
for latent
space

now
latent
vector

becomes input
to cls
model



$$\text{loss function} = RE + CE$$